DISPATCH PLANNING ERP

Senior Engineering Handover & Technical Blueprint

Audience: Frontend & Backend Engineering Teams | Status: Production-Ready (V6 Release)

1. Executive Summary & Tech Stack

This project is an Enterprise-Grade Dispatch Planning and Logistics ERP System built for managing supply chain workflows, vehicle fleet scheduling, multi-drop freight allocations, Proforma Invoice (PI) approvals, gate passes, incident rollbacks, PO Pickups, and Special Supply Chain Requests.

Layer	Technology / Tool	Key Purpose & Responsibility
Frontend Core	React 19, TypeScript 5.8, Vite 6	Modern type-safe SPA with fast HMR and optimized builds
Styling & UI	Tailwind CSS v4, Lucide React Icons	Utility-first responsive styling with enterprise color scheme
State & Mock API	Singleton Pub/Sub + LocalStorage (api.ts)	In-memory mock API ready for 1-to-1 REST endpoint replacement
Routing	Hash / Popstate Custom SPA Router (App.tsx)	Zero external routing dependency; fully compatible with GitHub Pages
Deployment	GitHub Pages (gh-pages branch)	Automated production builds with SPA 404.html fallback routing

2. Architecture & Data Flow (How It Works)

1. Single Data Service Pattern (api.ts): All operations go through dispatchService (Singleton in src/services/api.ts). It maintains in-memory state initialized from LocalStorage (or mockData.ts if empty). When any state change occurs, it persists to LocalStorage and triggers subscribers (pub/sub) so all active views update simultaneously without reload.
2. Centralized Modal System (AppLayout.tsx): All modal states (Create Dispatch, New PI, Gate Pass, Rollback, Special Request) are owned by AppLayout.tsx and exposed via helper callbacks (helpers.openNewDispatch, etc.) to child pages.
3. Route Architecture (App.tsx): Routing uses window.location.hash. The renderPage(helpers) function acts as the root switchboard.

3. Priority Files to Inspect First (Quick Onboarding)

If a new engineer (Frontend or Backend) joins today, they should read these 5 files in exact order:

Priority	File Path	Why It Matters	What to Check / Modify
1 (Critical)	**src/types/index.ts**	Central data contracts for the entire ERP system.	Review all schemas: ProformaInvoice, DispatchPlan, Vehicle, POPickup, SpecialRequest.
2 (Critical)	**src/services/api.ts**	Business logic, CRUD operations, state persistence, mock endpoints.	Backend team will replace these mock methods with real Axios / Fetch API endpoints.
3 (High)	**src/layouts/AppLayout.tsx & src/App.tsx**	Application shell, route switchboard, global modal registry.	Where new pages, navigation paths, and modals are registered.
4 (High)	**src/components/modals/CreateDispatchModal.tsx**	Core dispatch engine with vehicle capacity & weight calculation.	Inspect vehicle payload validation, multi-PI multi-drop selection logic.
5 (Medium)	**src/pages/DashboardPage.tsx**	Main executive control tower with real-time KPI metrics.	KPI cards, active trip queue, pending approval lists, quick action triggers.

4. Comprehensive File Inventory & Purpose Breakdown

A. Core Engine & State Layer

File Name	Detailed Working & Responsibility
src/types/index.ts	Contains all TypeScript interfaces, enum values, and data models (DispatchPlan, ProformaInvoice, Vehicle, Driver, Client, Warehouse, RollbackLog, POPickup, SpecialRequest).
src/services/api.ts	The Central DispatchDataService class. Manages all CRUD actions, state reactivity via pub/sub listeners, LocalStorage read/write, and helper status updaters.
src/data/mockData.ts	Initial seed data for Vehicles, Drivers, Clients, PIs, Dispatches, POPickups, and Special Requests when LocalStorage is empty.

B. Shell, Layout & Navigation

File Name	Detailed Working & Responsibility
src/App.tsx	Root React component. Handles hash-based routing, URL synchronization, and renders active page with modal helper bindings.
src/layouts/AppLayout.tsx	Master application wrapper containing Sidebar, Navbar, and all global modal dialogs. Provides helper callbacks to children.
src/components/Navbar.tsx	Top navigation bar showing Hub badge (Bhiwandi Hub), notification drawer, quick action buttons, and user profile.
src/components/Sidebar.tsx	Left sidebar with 5 primary nav items + collapsible 'More Operations' dropdown (Special_Request, PO-Pickup, Export Transfer, KB Orders).

C. Operational Pages (src/pages/)

File Name	Detailed Working & Responsibility
DashboardPage.tsx	Executive command center. Features 6 KPI cards (Pending PI, High Priority, Pending PO, Dispatches, Fleet Available, Special Requests), active loading queue, and pending approvals.
DispatchPlanningPage.tsx	Primary trip planning page. Interactive vehicle selector, capacity gauge, PI checklist, and multi-PI trip creation.
PendingPIPage.tsx	Registry of pending Proforma Invoices with Priority filter, approval workflow modal, and batch dispatch planning.
SpecialRequestPage.tsx	Web form registry for supply chain work orders. Displays Vendor Name, Location, Contact, Items, Deadline, Feedback, Attachments, and 'Mark Work Done' button.
POPickupPage.tsx	Purchase Order pickup registry. Features Planned vs Actual date tracking with automatic PENDING / COMPLETED status updates.
TodaysPlanningPage.tsx	Real-time dispatch monitor for dispatches scheduled today. View gate pass, trigger rollback, update trip status.
UpcomingPlanningPage.tsx	Forward-looking trip calendar for upcoming dispatches.
CompletedDispatchPage.tsx	Registry of completed deliveries with POD (Proof of Delivery) tracking.
DispatchHistoryPage.tsx	Historical searchable log of all dispatched trips.
RollbackPage.tsx	Audit trail of rolled-back / cancelled dispatch plans with mandatory rollback reason tracking.
ReportsPage.tsx	Fleet efficiency metrics, driver trip counts, volume statistics.
SettingsPage.tsx	Master data configuration: Fleet vehicles, drivers, clients, and warehouse management.
ExportTransferPage.tsx & KBOrdersPage.tsx	Specialized logistics views for export transfers and KB order handling.

D. Interactive Modals (src/components/modals/)

File Name	Detailed Working & Responsibility
CreateDispatchModal.tsx	Multi-PI dispatch planning modal with vehicle auto-calculation, weight/volume validation, multi-drop stop sequence ordering.

NewPIModal.tsx	Invoice creation form with multi-line item entry, GST calculation, destination warehouse, and priority selection.
SpecialRequestModal.tsx	Dedicated form for supply chain tasks: Vendor name, GPS location, contact info, item details, deadline date, feedback, and file attachments.
GatePassModal.tsx	Official security gate pass with vehicle barcode, driver details, seal number, and print-ready format.
RollbackModal.tsx	Trip rollback dialog enforcing audit reasoning before resetting allocated PIs back to pending status.
ApprovePIModal.tsx	Financial & logistics approval dialog for pending invoices.
ViewPIDetailsModal.tsx	Read-only detailed view of invoice line items, delivery location, and status history.

E. Common UI Widgets (src/components/common/)

File Name	Detailed Working & Responsibility
CapacityBar.tsx	Progress bar showing weight and volume capacity utilization percentages (Green < 80%, Yellow < 95%, Red >= 95%).
StatusBadge.tsx	Color-coded status badge supporting all ERP statuses (PENDING, APPROVED, LOADING, DISPATCHED, COMPLETED, ROLLED_BACK, etc.).
StatCard.tsx	Reusable KPI stat metric card component.

5. Code Audit & Customization Guide (What is Safe to Modify/Remove)

Category	Target Files	Engineering Guidelines
Backend API Migration	**src/services/api.ts**	Replace LocalStorage methods with async HTTP calls (Axios/Fetch). Frontend UI components will continue functioning seamlessly if method signatures are maintained.
Safe to Remove / Customize	**src/data/mockData.ts** **src/pages/ExportTransferPage.tsx** **src/pages/KBOrdersPage.tsx**	Mock data can be replaced with API responses. ExportTransfer & KBOrders are template pages ready for client-specific business rules or removal.
DO NOT Modify Without Review	**src/components/modals/CreateDispatchModal.tsx** **src/types/index.ts**	Vehicle payload calculation, stop sequencing, and PI allocation algorithms are tightly coupled with the dispatch workflow.
Hub & Location Config	**src/components/Navbar.tsx** **src/pages/DashboardPage.tsx**	Hub name ('Bhiwandi Hub') is currently hardcoded in header badges. Can be made dynamic from user profile/auth context.

6. Backend Team Integration Blueprint (API Contracts)

To connect the Frontend to a live Backend (Node.js / Spring Boot / Python / Go), implement these endpoints:

Method	Endpoint Route	Frontend Consumer Method	Payload / Response Details
GET	**/api/v1/pis**	dispatchService.getPIs()	Returns list of Proforma Invoices
POST	**/api/v1/pis**	dispatchService.addPI(piData)	Creates new PI with line items
PATCH	**/api/v1/pis/:id/status**	dispatchService.updatePIStatus(id, status)	Approve, reject, or mark dispatched
GET	**/api/v1/dispatches**	dispatchService.getDispatchPlans()	Returns all dispatch plans with stops & vehicles
POST	**/api/v1/dispatches**	dispatchService.createDispatchPlan(plan)	Creates trip, reserves vehicle, marks PIs planned
POST	**/api/v1/dispatches/:id/rollback**	dispatchService.rollbackDispatch(id, reason)	Rolls back trip, releases vehicle, resets PIs to PENDING

GET	**/api/v1/fleet/vehicles**	dispatchService.getVehicles()	Returns vehicle fleet with capacity & availability
GET	**/api/v1/fleet/drivers**	dispatchService.getDrivers()	Returns driver master data
GET	**/api/v1/special-requests**	dispatchService.getSpecialRequests()	Returns supply chain special work requests
POST	**/api/v1/special-requests**	dispatchService.addSpecialRequest(req)	Creates new special request with attachments
PATCH	**/api/v1/special-requests/:id/done**	dispatchService.updateSpecialRequestStatus(id, 'WORK_DONE')	Marks request as work completed